2.4.9 Operator Precedence Grammar

Format: (1) No 2 or more variable side by side
(2) No $\in$ production.

Example:

$E \to E + T \mid T$
$T \to T * F \mid F$
$F \to (E) \mid id$
O.G.

or

$E \to E + E$
$E \to E * E$
$E \to a / b$
O.G

or

$S \to aSa \mid bSb \mid a \mid b$
O.G

Or

$S \to AB$
$A \to aA \mid \in$
$B \to bB \mid \in$

Not O.G

2.2.10 Checking LL (1) without Table

$A \to \alpha_1 \mid \alpha_1 \mid \alpha_1$ then $\to$

first $(\alpha_1) \cap$ first $(\alpha_2) = \varnothing$
first $(\alpha_1) \cap$ first $(\alpha_3) = \varnothing$
first $(\alpha_2) \cap$ first $(\alpha_3) = \varnothing$

$A \to \alpha_1 \mid \alpha_2 \mid \alpha_3 \mid \in$

first $(\alpha_1) \cap$ first $(\alpha_2) = \varnothing$
first $(\alpha_1) \cap$ first $(\alpha_3) = \varnothing$
first $(\alpha_2) \cap$ first $(\alpha_3) = \varnothing$
follow $(A) \cap$ first $(\alpha_1) = \varnothing$
follow $(A) \cap$ first $(\alpha_2) = \varnothing$
follow $(A) \cap$ first $(\alpha_3) = \varnothing$

2.2.11 Bottom-UP Parser

- It uses RMD in reverse and has no problem with:
  (a) Left recursion (b) Common Prefix
  SLR (1) LR (0) CLR (1) LR (1)
  LR (0) items LALR (1) items

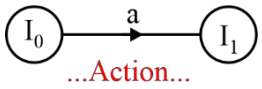
  LR (1) = LR(0) +1 Lookahead
- No parser possible for ambiguous grammar.
- There are some unambiguous grammars for which, there are no Parser.

2.2.12 Basic Algorithm for Construction

- Augment the grammar and expand it, and give numbers to it
- Construct LR (0) or LR (1) item set.
- From that fill the entries in the Table Accordingly.

2.2.13 Types of Entries

(1) Shift Entries: Transitions or Terminals



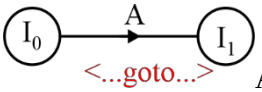
Entry:

	a
I_0	S_1

$a \in$ Terminal I_0, I_1: Item sets.

- Shift entries are some for all Bottom – up Parser.

(2) State Entry: Transition or non – terminal (variable)



$A \rightarrow$ Variable

Entry:

	A
I_0	1

- Same for all Bottom-up Parser.

(3) Reduce Entity: done for each separate production in the item set of type:

$i > \times \rightarrow \alpha$

where, $i \rightarrow$ Production Number

$\times \rightarrow$ Producing Variable

$\alpha \rightarrow$ Grammar String

(a) LR (0) Parser:

Put Ri in every cell of the set-in action table **(ALL).**

(b) SLR (1) Parser:

Put Ri only in the follow(x) form the Grammar. **(Follow(x))**

(c) LALR (1) and CLR (1):

Put Ri only in the look ahead of the production **(Lookaheads)**

2.2.14 Conflicts

LR (0) Parser:

SR: Shift Reduce Conflict

$S \rightarrow \quad X \rightarrow \alpha \cdot t\beta \quad \Rightarrow$ then SR

$R \rightarrow Y \rightarrow \delta \cdot$ conflict

RR: Reduce Reduce conflict

$R \rightarrow X \rightarrow \alpha \cdot \quad \Rightarrow$ then RR

$R \rightarrow X \rightarrow \beta \cdot$

SLR (1) Parser:

SR:

$S \rightarrow X \rightarrow \alpha \cdot t\beta$
$R \rightarrow Y \rightarrow \delta \cdot$
$t \in$ follow (Y)
} $ ⇒then SR, conflict

RR:

$X \rightarrow \alpha \cdot$
$Y \rightarrow \beta \cdot$
} $ ⇒then RR

Follow (x)n follow (y) $\neq \varnothing$

LALR (1) and CLR (1): Same as SLR (1), but instead

SR:

$X \rightarrow \alpha \cdot t\, \beta \cdot L_1$
$Y \rightarrow \delta \cdot L_2$
} $⇒then SR, conflict

$T \in L_2$

RR:

$X \rightarrow \alpha \cdot, L_1$
$Y \rightarrow \beta \cdot, L_2$
} $ then RR, conflict

$L_1 \cap L_2 \neq \varnothing$

2.4.15 Inadequate Static

A static having ANY conflict is called a conflicting state or independent state.

Note:
The state S' → S. or S' → S.,$ is accepted state, and this is not a reduction.

- The only difference between CLR (1) and LALR (1) is that, the states with the similar items, but different Lookaheads are merged together to Reduce space.

Important Points

1. If CLR (1) doesn't have any conflict, then conflict may or may not arise after merging in LALR (1).
2. If LALR (1) has SR – conflict, then we can conclude that CLR (1) also has SR – Conflict.
3. LALR (1) has SR – conflict if and only if CLR (1) also has SR.

- We can construct parser for every unambiguous regular grammar: [CLR (1) Parser].

	L Left to right scan of i/p	R Using reverse RMD	(0) No Lookhead	
S Simple	L L to R Scan	R Using Reverse RMD	(1) Lookahead	
L Look	A Ahead	L L to R Scan	R Revers RMD	(1) Lookahead
C Canonical	L L to R Scan	R reverse RMD	(1) Lookahead	

Very Important Point:

LALR (1) Parser can Parse non LALR (1) grammar, when only non-SR – Conflict by favouring shift over reduce.

Example: $E \rightarrow E + E \mid E * E \mid id \mid 2 + 3 * 5 \Rightarrow E + E \cdot * 5$

❑❑❑